Assignment 3

Q1 Create 1 Public Docker Hub registry named flask_webapp_yourname.

● Clone below repository on your system.
https://github.com/mmumshad/simple-webapp-docker.git
● Initialize a local repository & copy the code from above repo to your local repository in your working branch.
● Once code is copied to the local repository, build the docker image & add meaningful tags with version 1 and push to Docker Hub registry.
● Once Images are pushed to Docker hub registries, create a directory named kube. Inside the kube directory create deployement.yaml file with 3 replication , labels app: flask-webapp , containerPort: 8080 and add the image that you have pushed in Docker Hub registry.
● Create one service.yaml file with type nodeport & select flask-webapp with port 8080 & targetPort 8080 with any nodePort between range 30000-32768.
● Once a service is created try accessing the web page in the browser as below. (30011 is nodeport mentioned in service.yaml). Meanwhile open app.py from your code to understand paths & output.
http://master_ip:30011/
http://master_ip:30011/how are you
● Now , update the app.py from your code and add below route above if **name** == "**main**" line
@app.route('/Who are you')
def cloudethix():
return 'Yes, I am cloudethix, and You !!!'
● Once the file is updated , rebuild the docker image & add meaningful tags with version 2 and push to Docker Hub registry.
● Now we have the latest docker image in repo, It's time to roll out a new image. Roll out the new Image with all three ways one by one.

1. With kubectl set command
2. With kubectl edit deployment
3. With deployment.yaml file modification.
   - Run the # kubectl rollout command to check status and history.
   - Note:- Once above step 1 is done , run # kubectl rollout undo deployment command to rollback the change and then try a second way of rollout.
   - In the browser run all three routes & notice the changes.

http://master_ip:30011/
http://master_ip:30011/how are you
http://master_ip:30011/Who are you

- Once done with all above steps , commit all the changes to the remote repository

System Output

```
avej_lanjekar@AvejLanjekar:~git clone https://github.com/mmumshad/simple-
webapp-docker.git
Cloning into 'simple-webapp-docker'...
remote: Enumerating objects: 14, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 14 (delta 3), reused 2 (delta 2), pack-reused 7 (from 1)
Receiving objects: 100% (14/14), done.
Resolving deltas: 100% (3/3), done.

avej_lanjekar@AvejLanjekar:~ cp -r simple-webapp-docker/* flask_webapp/

avej_lanjekar@AvejLanjekar:~ ls
flask_webapp  simple-webapp-docker

avej_lanjekar@AvejLanjekar:~ cd flask_webapp/

avej_lanjekar@AvejLanjekar:~ ls
Dockerfile  app.py

avej_lanjekar@AvejLanjekar:~ git init
hint: Using 'master' as the name for the initial branch. This default branch
name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this
warning,
hint: call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName
false"
Initialized empty Git repository in /home/avej_lanjekar/kubernetes-
```

```
Assignment/Repetition-1/Assignment3/Q1/flask_webapp/.git/

avej_lanjekar@AvejLanjekar:~ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Dockerfile
        app.py

nothing added to commit but untracked files present (use "git add" to track)

avej_lanjekar@AvejLanjekar:~ docker build -t flask_webapp:version1 .
STEP 1/5: FROM ubuntu:20.04
Resolved "ubuntu" as an alias
(/etc/containers/registries.conf.d/shortnames.conf)
Trying to pull docker.io/library/ubuntu:20.04...
Getting image source signatures
Copying blob 13b7e930469f done   |
Copying config b7bab04fd9 done   |
Writing manifest to image destination
STEP 2/5: RUN apt-get update && apt-get install -y python3 python3-pip
Get:1 http://security.ubuntu.com/ubuntu focal-security InRelease [128 kB]
Get:2 http://archive.ubuntu.com/ubuntu focal InRelease [265 kB]
Get:3 http://security.ubuntu.com/ubuntu focal-security/multiverse amd64
Packages [32.5 kB]
Get:4 http://security.ubuntu.com/ubuntu focal-security/restricted amd64
Packages [4710 kB]
Get:5 http://archive.ubuntu.com/ubuntu focal-updates InRelease [128 kB]
Get:6 http://archive.ubuntu.com/ubuntu focal-backports InRelease [128 kB]
Get:7 http://security.ubuntu.com/ubuntu focal-security/universe amd64 Packages
[1300 kB]
Get:8 http://security.ubuntu.com/ubuntu focal-security/main amd64 Packages
[4368 kB]
Get:9 http://archive.ubuntu.com/ubuntu focal/multiverse amd64 Packages [177
kB]
Get:10 http://archive.ubuntu.com/ubuntu focal/universe amd64 Packages [11.3
MB]

avej_lanjekar@AvejLanjekar:~ docker tag flask_webapp:version1
avejlanjekar45/flask_webapp_avej:version1


avej_lanjekar@AvejLanjekar:~ docker push
```

```
avejlanjekar45/flask_webapp_avej:version1
Getting image source signatures
Copying blob 68199f7ab160 done   |
Copying blob 96a1aa32f272 done   |
Copying blob 7ef1324fc058 done   |
Copying blob 13b7e930469f skipped: already exists
Copying config 0d76e83579 done   |
Writing manifest to image destination

avej_lanjekar@AvejLanjekar:~ mkdir kube

avej_lanjekar@AvejLanjekar:~ cd kube/

avej_lanjekar@AvejLanjekar:~ vim deployment.yml
avej_lanjekar@AvejLanjekar:~ ls
deployment.yml
avej_lanjekar@AvejLanjekar:~ cat deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-webapp
  namespace: ns-1
spec:
  replicas: 3
  selector:
    matchLabels:
      app: flask-webapp
  template:
    metadata:
      labels:
        app: flask-webapp
    spec:
      containers:
        - name: flask-webapp
          image: avejlanjekar45/flask_webapp_avej:version-2
          ports:
            - containerPort: 8080

avej_lanjekar@AvejLanjekar:~ vim service.yml
avej_lanjekar@AvejLanjekar:~ cat service.yml
apiVersion: v1
kind: Service
metadata:
  name: flask-webapp-svc
  namespace: ns-1
spec:
```

```
  type: NodePort
  selector:
    app: flask-webapp
  ports:
    - port: 8080
      targetPort: 8080
      nodePort: 30011

avej_lanjekar@AvejLanjekar:~ kubectl get deployments
NAME           READY   UP-TO-DATE   AVAILABLE   AGE
flask-webapp   3/3     3            3           3h15m
nfs-server     1/1     1            1           8h

avej_lanjekar@AvejLanjekar:~ kubectl get svc
NAME                    TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)
AGE
cloudethix-clusterip   ClusterIP   10.97.252.69     <none>        80/TCP
5h14m
flask-webapp-svc       NodePort    10.104.135.92    <none>
8080:30011/TCP   3h14m
inventory-service      ClusterIP   10.102.75.180    <none>        5678/TCP
8h
nfs-server             ClusterIP   10.109.166.116   <none>        2049/TCP
8h

avej_lanjekar@AvejLanjekar:~ curl http://192.168.49.2:30011/
Welcome!

avej_lanjekar@AvejLanjekar:~ curl "http://192.168.49.2:30011/how%20are%20you"
I am good, how about you?

avej_lanjekar@AvejLanjekar:~ cat app.py
import os
from flask import Flask
app = Flask(__name__)

@app.route("/")
def main():
    return "Welcome!"

@app.route('/how are you')
def hello():
    return 'I am good, how about you?'

if __name__ == "__main__":
    app.run()
```

```
@app.route('/Who are you')
def cloudethix():
    return 'Yes, I am cloudethix, and You !!!'

avej_lanjekar@AvejLanjekar:~ docker build -t flask_webapp:version-2 .
STEP 1/5: FROM ubuntu:20.04
STEP 2/5: RUN apt-get update && apt-get install -y python3 python3-pip
--> Using cache
a2753bbdc460a9c05a7240ec9f1be9eaf3732ba6d592b3a403b3ee2b81a82cbf
--> a2753bbdc460
STEP 3/5: RUN pip install flask
--> Using cache
78efa4ef97b780ce79b296f086decd52be0646fd98d8aaf4667f6322cedc4436
--> 78efa4ef97b7
STEP 4/5: COPY app.py /opt/
--> acf77daebf8f
STEP 5/5: ENTRYPOINT FLASK_APP=/opt/app.py flask run --host=0.0.0.0 --
port=8080
COMMIT flask_webapp:version-2
--> f62c9c6e657b
Successfully tagged localhost/flask_webapp:version-2
f62c9c6e657bbac895b6de5dd6a831b41298094116dc27c5bb69ad3977a3910f

avej_lanjekar@AvejLanjekar:~ docker tag flask_webapp:version-2
avejlanjekar45/flask_webapp_avej:version-2

avej_lanjekar@AvejLanjekar:~ docker push
avejlanjekar45/flask_webapp_avej:version-2
Getting image source signatures
Copying blob 0cb734515e39 done   |
Copying blob 67f28aede3f8 skipped: already exists
Copying blob 13b7e930469f skipped: already exists
Copying blob c6459ed4af3c skipped: already exists
Copying config f62c9c6e65 done   |
Writing manifest to image destination

avej_lanjekar@AvejLanjekar:~ kubectl set image deployment/flask-webapp flask-
webapp=avejlanjekar45/flask_webapp_avej:version-2
deployment.apps/flask-webapp image updated

avej_lanjekar@AvejLanjekar:~ kubectl rollout status deployment/flask-webapp
deployment "flask-webapp" successfully rolled out

avej_lanjekar@AvejLanjekar:~ curl http://192.168.49.2:30011/
Welcome!
```

```
avej_lanjekar@AvejLanjekar:~ curl http://192.168.49.2:30011/how%20are%20you
I am good, how about you?

avej_lanjekar@AvejLanjekar:~ curl "http://192.168.49.2:30011/Who%20are%20you"
Yes, I am cloudethix, and You !!!

avej_lanjekar@AvejLanjekar:~ kubectl rollout history deployment/flask-webapp
deployment.apps/flask-webapp
REVISION  CHANGE-CAUSE
1         <none>
2         <none>

avej_lanjekar@AvejLanjekar:~ kubectl edit deployment/flask-webapp
deployment.apps/flask-webapp edited

avej_lanjekar@AvejLanjekar:~ kubectl rollout status deployment/flask-webapp
deployment "flask-webapp" successfully rolled out

avej_lanjekar@AvejLanjekar:~ kubectl rollout history deployment/flask-webapp
deployment.apps/flask-webapp
REVISION  CHANGE-CAUSE
2         <none>
3         <none>

avej_lanjekar@AvejLanjekar:~ kubectl apply -f deployment.yml
deployment.apps/flask-webapp configured

avej_lanjekar@AvejLanjekar:~ kubectl rollout status deployment/flask-webapp
deployment "flask-webapp" successfully rolled out

avej_lanjekar@AvejLanjekar:~ kubectl rollout history deployment/flask-webapp
deployment.apps/flask-webapp
REVISION  CHANGE-CAUSE
3         <none>
4         <none>

avej_lanjekar@AvejLanjekar:~ curl "http://192.168.49.2:30011/how%20are%20you"
I am good, how about you?

avej_lanjekar@AvejLanjekar:~ curl "http://192.168.49.2:30011/Who%20are%20you"
Yes, I am cloudethix, and You !!!

avej_lanjekar@AvejLanjekar:~ curl "http://192.168.49.2:30011/"
Welcome!
```

```
avej_lanjekar@AvejLanjekar:~ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Dockerfile
        app.py
        kube/

nothing added to commit but untracked files present (use "git add" to track)


avej_lanjekar@AvejLanjekar:~ git add .

avej_lanjekar@AvejLanjekar:~ git commit -m "Added kubernetes deployment"
[master (root-commit) e6006b7] Added kubernetes deployment
 4 files changed, 60 insertions(+)
 create mode 100644 Dockerfile
 create mode 100644 app.py
 create mode 100644 kube/deployment.yml
 create mode 100644 kube/service.yml

avej_lanjekar@AvejLanjekar:~ git status
On branch master
nothing to commit, working tree clean


avej_lanjekar@AvejLanjekar:~ git remote add origin
git@github.com:Avejlanjekar/flask-k8s-rolling-update-deployment.git

avej_lanjekar@AvejLanjekar:~ git branch -M main

avej_lanjekar@AvejLanjekar:~ git push -u origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 1.05 KiB | 269.00 KiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:Avejlanjekar/flask-k8s-rolling-update-deployment.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

2 →

- Download and install Lens & access your k8s cluster from Lens.
- Create nginx Pod and Nodeport service. Check the Pod logs from Lens.
- Check the service from lens. Also login to the pod shell using the lens.
- Take snaps and delete the resources that you have just created.

```
avej_lanjekar@AvejLanjekar:Q4$ minikube dashboard
🔌  Enabling dashboard ...
    ▪ Using image docker.io/kubernetesui/dashboard:v2.7.0
    ▪ Using image docker.io/kubernetesui/metrics-scraper:v1.0.8
💡  Some dashboard features require the metrics-server addon. To enable all
features please run:

        minikube addons enable metrics-server

🤔  Verifying dashboard health ...
🚀  Launching proxy ...
🤔  Verifying proxy health ...
🎉  Opening http://127.0.0.1:43941/api/v1/namespaces/kubernetes-
dashboard/services/http:kubernetes-dashboard:/proxy/ in your default
browser...
👉  http://127.0.0.1:43941/api/v1/namespaces/kubernetes-
dashboard/services/http:kubernetes-dashboard:/proxy/

avej_lanjekar@AvejLanjekar:Q2$ ls
pod.yml  service.yml

avej_lanjekar@AvejLanjekar:Q2$ cat pod.yml
apiVersion: v1
kind: Pod
metadata:
  name: pod-nginx
  labels:
    app: nginx
spec:
  containers:
    - name: nginx
      image: nginx
      ports:
        - containerPort: 80

avej_lanjekar@AvejLanjekar:Q2$ cat service.yml
apiVersion: v1
kind: Service
metadata:
  name: nodeport-svc
```

```
spec:
  selector:
    app: nginx
  type: NodePort
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30007
```

Q3 Create 1 Public Docker Hub registry named cloudethix_configmap_yourname.

● Clone below repository on your system.
https://github.com/zembutsu/docker-sample-nginx.git
● Initialize a local repository & copy the code from above repo to your local repository in the working branch.
● Once code is copied , build a docker image from docker file and add meaningful tags and push to docker hub repository.
● Once Images are pushed to Docker hub registries, create a directory named kube. Inside the kube directory create deployement.yaml file with 3 replication , labels app: frontend-webapp , containerPort: 80 and add the image that you have pushed in Docker Hub registry.
● Create one service.yaml file with type nodeport & select frontend-webapp pod with port 80 & targetPort 80 with any nodePort between range 30000-32768.
● Once the service is created try accessing the web page in the browser as below. Notice the changes & take the snap.
● Now create a configmap.yaml file with below data & delete the deployment that you have created.

```
<html>
<body>
<h1> I am Cloudethix Team, Are you ?!! </h1>
Version: 1.1
</body>
</html>
```

● Then update the same deployment.yaml file and mount configmap as volume on container using volumeMounts with mountPath /usr/share/nginx/html/

● Now it's time to create configmap & deployment. Once created , try to access the webpage in the browser & confirm that the index page is the same as we have in configmap.

```
avej_lanjekar@AvejLanjekar:~ git clone https://github.com/zembutsu/docker-sample-nginx.git
Cloning into 'docker-sample-nginx'...
remote: Enumerating objects: 22, done.
remote: Counting objects: 100% (12/12), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 22 (delta 7), reused 6 (delta 6), pack-reused 10 (from 1)
Receiving objects: 100% (22/22), done.
Resolving deltas: 100% (7/7), done.

avej_lanjekar@AvejLanjekar:~ mkdir clouethix-configmap

avej_lanjekar@AvejLanjekar:~ ls
clouethix-configmap  docker-sample-nginx

avej_lanjekar@AvejLanjekar:~ cp docker-sample-nginx/* clouethix-configmap/

avej_lanjekar@AvejLanjekar:~ ls
clouethix-configmap  docker-sample-nginx

avej_lanjekar@AvejLanjekar:~ cd clouethix-configmap/

avej_lanjekar@AvejLanjekar:~ ls
Dockerfile  LICENSE  README.md  default.conf  index.html

avej_lanjekar@AvejLanjekar:~ docker build -t cloudethix-configmap .
STEP 1/3: FROM nginx:alpine
STEP 2/3: COPY default.conf /etc/nginx/conf.d/
--> Using cache
0989e7142eba649abc4d95f1780a91bf1f19479e0fc6153f85667cfc7b500de5
--> 0989e7142eba
STEP 3/3: COPY index.html /usr/share/nginx/html/
COMMIT cloudethix-configmap
--> b8bf8728f7f5
Successfully tagged localhost/cloudethix-configmap:latest
b8bf8728f7f56726509964c82b8020ce24f4c0cd049539aeb69c604660d03136

avej_lanjekar@AvejLanjekar:~ docker tag cloudethix-configmap
avejlanjekar45/cloudethix_configmap_avej

avej_lanjekar@AvejLanjekar:~ docker push
avejlanjekar45/cloudethix_configmap_avej
```

```
Getting image source signatures
Copying blob 1ed8e39a7434 skipped: already exists
Copying blob 8cdfe4f23778 skipped: already exists
Copying blob d94291c26261 skipped: already exists
Copying blob 0ea935727878 skipped: already exists
Copying blob 0d62d88506ba skipped: already exists
Copying blob 55afa1ecc21d skipped: already exists
Copying blob 03c47411bc64 done   |
Copying blob da4dea1b00af skipped: already exists
Copying blob fb597529c916 skipped: already exists
Copying blob ba05887de274 skipped: already exists
Copying config b8bf8728f7 done   |
Writing manifest to image destination

avej_lanjekar@AvejLanjekar:~ cat deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cloudethix-configmap
  namespace: ns-1
spec:
  replicas: 3
  selector:
    matchLabels:
      app: frontend-webapp
  template:
    metadata:
      labels:
        app: frontend-webapp
    spec:
      containers:
        - name: cloudethix-cm
          image: avejlanjekar45/cloudethix_configmap_avej
          ports:
            - containerPort: 80
          volumeMounts:
            - name: frontend-config-map
              mountPath: /usr/share/nginx/html/
      volumes:
        - name: frontend-config-map
          configMap:
            name: frontend-config


avej_lanjekar@AvejLanjekar:~ cat service.yaml
apiVersion: v1
```

```
kind: Service
metadata:
  name: cloudethix-configmap-svc
  namespace: ns-1
spec:
  type: NodePort
  selector:
    app: frontend-webapp
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30010

avej_lanjekar@AvejLanjekar:~ curl http://192.168.49.2:30010
<html>
<body>
        <h1>Host: cloudethix-configmap-7b8849c65f-d4464</h1>
        Version: 1.1
</body>
</html>

avej_lanjekar@AvejLanjekar:~ kubectl delete deployment cloudethix-configmap
deployment.apps "cloudethix-configmap" deleted from ns-1 namespace

avej_lanjekar@AvejLanjekar:~ cat configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: frontend-config
data:
  index.html: |
    <html>
    <body>
    <h1>I am Cloudethix Team, Are you ?!!</h1>
    Version: 1.1
    </body>
    </html>

avej_lanjekar@AvejLanjekar:~ kubectl apply -f configmap.yaml
configmap/frontend-config created

avej_lanjekar@AvejLanjekar:~ cat deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cloudethix-configmap
```

```
  namespace: ns-1
spec:
  replicas: 3
  selector:
    matchLabels:
      app: frontend-webapp
  template:
    metadata:
      labels:
        app: frontend-webapp
    spec:
      containers:
        - name: cloudethix-cm
          image: avejlanjekar45/cloudethix_configmap_avej
          ports:
            - containerPort: 80
          volumeMounts:
            - name: frontend-config-map
              mountPath: /usr/share/nginx/html/
      volumes:
        - name: frontend-config-map
          configMap:
            name: frontend-config


avej_lanjekar@AvejLanjekar:~ kubectl get deployments
NAME                   READY   UP-TO-DATE   AVAILABLE   AGE
cloudethix-configmap   3/3     3            3           5m18s

avej_lanjekar@AvejLanjekar:~ kubectl get cm
NAME               DATA   AGE
frontend-config    1      5m53s

avej_lanjekar@AvejLanjekar:~ curl http://192.168.49.2:30010
<html>
<body>
<h1>I am Cloudethix Team, Are you ?!!</h1>
Version: 1.1
</body>
</html>
```

Que 4 →

- Create 1 Public Docker Hub registry named cloudethix_multicontainer_yourname.
- Clone below repository on your system.

https://github.com/janakiramm/Kubernetes-multi-container-pod.git

● Initialize a local repository & copy the code from above repo to your local repository in any of your working branches.

● Once code is copied , go to the Build directory and build docker image from docker file and add meaningful tags and push to docker hub repository.

● Now go to the deploy directory and notice the files.

● Here, web-pod-1.yml file will create the pod with two containers (Multi container). Take a note of lables , name of containers and ports. Also, please make sure you will update the python container image that you have pushed to your docker registry.

● Now, open web-svc.yml file and notice service Type , selectors & targetPort. Apply the file.

● Now open db-pod.yml & notice the lables , name , Image, containerPort and apply the file.

● Now open the db-svc.yml file and notice service Type , selectors & targetPort. Apply the file.

● Once above files are applied , Check that the Pods and Services are created using command line or lens.

● Now , from the command line run below urls & notice the changes.

curl http://$NODE_IP$:NODE_PORT/init

Initialize the database with sample schema

● Now it's time to Insert some sample data. Make sure you will use correct $NODE_IP$:NODE_PORT

curl -i -H "Content-Type: application/json" -X POST -d '{"uid": "1", "user":"John Doe"}' http://$NODE_IP$:NODE_PORT/users/add

curl -i -H "Content-Type: application/json" -X POST -d '{"uid": "2", "user":"Jane Doe"}' http://$NODE_IP$:NODE_PORT/users/add

curl -i -H "Content-Type: application/json" -X POST -d '{"uid": "3", "user":"Bill Colls"}' http://$NODE_IP$:NODE_PORT/users/add

curl -i -H "Content-Type: application/json" -X POST -d '{"uid": "4", "user":"Mike Taylor"}' http://$NODE_IP$:NODE_PORT/users/add

● Now access the data that we have added to database using below command.

curl http://$NODE_IP$:NODE_PORT/users/1

● The second time you access the data, it appends '(c)' indicating that it is pulled from the Redis cache.

● Also, try to access mysql shell i.e db pod & run select * from the users table. check app.py for DB related information.

● Prepare proper documentation in brief & write start to end flow. Refer below link if you face any issues.

https://github.com/janakiramm/Kubernetes-multi-container-pod

```
avej_lanjekar@AvejLanjekar:Deploy$ ls
db-pod.yml  db-svc.yml  web-pod-1.yml  web-pod-2.yml  web-rc.yml  web-svc.yml

avej_lanjekar@AvejLanjekar:Deploy$ cat web-pod-1.yml
apiVersion: "v1"
kind: Pod
metadata:
  name: web1
  labels:
    name: web
    app: demo
spec:
  containers:
    - name: redis
      image: redis
      ports:
        - containerPort: 6379
          name: redis
          protocol: TCP
    - name: python
      image: avejlanjekar45/cloudethix_multicontainer_avej
      env:
        - name: "REDIS_HOST"
          value: "localhost"
      ports:
        - containerPort: 5000
          name: http
          protocol: TCP

avej_lanjekar@AvejLanjekar:Deploy$ cat web-svc.yml
apiVersion: v1
kind: Service
metadata:
  name: web
  labels:
    name: web
    app: demo
spec:
  selector:
    name: web
  type: NodePort
  ports:
   - port: 80
     name: http
     targetPort: 5000
     protocol: TCP
```

```
avej_lanjekar@AvejLanjekar:Deploy$ kubectl apply -f web-pod-1.yml
pod/web1 created

avej_lanjekar@AvejLanjekar:Deploy$ kubectl apply -f web-svc.yml
service/web created

avej_lanjekar@AvejLanjekar:Deploy$ cat db-pod.yml
apiVersion: "v1"
kind: Pod
metadata:
  name: mysql
  labels:
    name: mysql
    app: demo
spec:
  containers:
    - name: mysql
      image: mysql:5.7.25
      ports:
        - containerPort: 3306
          protocol: TCP
      env:
        -
          name: "MYSQL_ROOT_PASSWORD"
          value: "password"

avej_lanjekar@AvejLanjekar:Deploy$ kubectl apply -f db-pod.yml
pod/mysql created

avej_lanjekar@AvejLanjekar:Deploy$ cat db-svc.yml
apiVersion: v1
kind: Service
metadata:
  name: mysql
  labels:
    name: mysql
    app: demo
spec:
  ports:
  - port: 3306
    name: mysql
    targetPort: 3306
  selector:
    name: mysql
    app: demo
```

```
avej_lanjekar@AvejLanjekar:Deploy$ kubectl apply -f db-svc.yml
service/mysql configured

avej_lanjekar@AvejLanjekar:Deploy$ minikube service web --url
http://127.0.0.1:46311
❗  Because you are using a Docker driver on linux, the terminal needs to be
open to run it.

avej_lanjekar@AvejLanjekar:~$ curl http://127.0.0.1:46311/init
DB Init done

avej_lanjekar@AvejLanjekar:~$ curl -i -H "Content-Type: application/json" -X
POST -d '{"uid": "2",
"user":"Jane Doe"}' http://127.0.0.1:46311/users/add
HTTP/1.0 200 OK
Content-Type: application/json
Content-Length: 5
Server: Werkzeug/1.0.1 Python/2.7.15
Date: Thu, 03 Sep 2026 05:42:48 GMT

Addedavej_lanjekar@AvejLanjekar:~$
avej_lanjekar@AvejLanjekar:~$ curl -i -H "Content-Type: application/json" -X
POST -d '{"uid": "3",
"user":"Bill Colls"}' http://127.0.0.1:46311/users/add
HTTP/1.0 200 OK
Content-Type: application/json
Content-Length: 5
Server: Werkzeug/1.0.1 Python/2.7.15
Date: Thu, 03 Sep 2026 05:43:13 GMT

avej_lanjekar@AvejLanjekar:~$ curl -i -H "Content-Type: application/json" -X
POST -d '{"uid": "4",
"user":"Mike Taylor"}' http://127.0.0.1:46311/users/add
HTTP/1.0 200 OK
Content-Type: application/json
Content-Length: 5
Server: Werkzeug/1.0.1 Python/2.7.15
Date: Thu, 03 Sep 2026 05:43:34 GMT

avej_lanjekar@AvejLanjekar:~$ curl http://127.0.0.1:46311/users/1
John Doe

avej_lanjekar@AvejLanjekar:~$ curl http://127.0.0.1:46311/users/1
John Doe(c)
```

```
avej_lanjekar@AvejLanjekar:~$ kubectl exec -it mysql -- mysql -uroot -
ppassword
mysql: [Warning] Using a password on the command line interface can be
insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 4
Server version: 5.7.25 MySQL Community Server (GPL)

Copyright (c) 2000, 2019, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| USERDB             |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.01 sec)

mysql> use information_schema
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql>
mysql> show tables;
+---------------------------------------+
| Tables_in_information_schema          |
+---------------------------------------+
| CHARACTER_SETS                        |
| COLLATIONS                            |
| COLLATION_CHARACTER_SET_APPLICABILITY |
| COLUMNS                               |
| COLUMN_PRIVILEGES                     |
| ENGINES                               |
| EVENTS                                |
| FILES                                 |
```

```
| GLOBAL_STATUS                         |
| GLOBAL_VARIABLES                      |
| KEY_COLUMN_USAGE                      |
| OPTIMIZER_TRACE                       |
| PARAMETERS                            |
| PARTITIONS                            |
| PLUGINS                               |
| PROCESSLIST                           |
| PROFILING                             |
| REFERENTIAL_CONSTRAINTS               |
| ROUTINES                              |
| SCHEMATA                              |
| SCHEMA_PRIVILEGES                     |
| SESSION_STATUS                        |
| SESSION_VARIABLES                     |
| STATISTICS                            |
| TABLES                                |
| TABLESPACES                           |
| TABLE_CONSTRAINTS                     |
| TABLE_PRIVILEGES                      |
| TRIGGERS                              |
| USER_PRIVILEGES                       |
| VIEWS                                 |
| INNODB_LOCKS                          |
| INNODB_TRX                            |
| INNODB_SYS_DATAFILES                  |
| INNODB_FT_CONFIG                      |
| INNODB_SYS_VIRTUAL                    |
| INNODB_CMP                            |
| INNODB_FT_BEING_DELETED               |
| INNODB_CMP_RESET                      |
| INNODB_CMP_PER_INDEX                  |
| INNODB_CMPMEM_RESET                   |
| INNODB_FT_DELETED                     |
| INNODB_BUFFER_PAGE_LRU                |
| INNODB_LOCK_WAITS                     |
| INNODB_TEMP_TABLE_INFO                |
| INNODB_SYS_INDEXES                    |
| INNODB_SYS_TABLES                     |
| INNODB_SYS_FIELDS                     |
| INNODB_CMP_PER_INDEX_RESET            |
| INNODB_BUFFER_PAGE                    |
| INNODB_FT_DEFAULT_STOPWORD            |
| INNODB_FT_INDEX_TABLE                 |
| INNODB_FT_INDEX_CACHE                 |
| INNODB_SYS_TABLESPACES                |
```

```
| INNODB_METRICS                         |
| INNODB_SYS_FOREIGN_COLS                |
| INNODB_CMPMEM                          |
| INNODB_BUFFER_POOL_STATS               |
| INNODB_SYS_COLUMNS                     |
| INNODB_SYS_FOREIGN                     |
| INNODB_SYS_TABLESTATS                  |
+----------------------------------------+
61 rows in set (0.00 sec)

mysql> select * from users;
ERROR 1109 (42S02): Unknown table 'users' in information_schema
mysql> use USERDB;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql>
mysql>
mysql> show tables;
+------------------+
| Tables_in_USERDB |
+------------------+
| users            |
+------------------+
1 row in set (0.00 sec)

mysql> select * from users;
+------+-------------+
| ID   | USER        |
+------+-------------+
|    1 | John Doe    |
|    2 | Jane Doe    |
|    3 | Bill Colls  |
|    4 | Mike Taylor |
+------+-------------+
4 rows in set (0.00 sec)

mysql>
```

Que 5 →

● Create 1 Public Docker Hub registry named cloudethix_Initcontainer_yourname.
● Clone below repository on your system.
https://github.com/janakiramm/simpleapp.git
● Initialize a local repository & copy the code from above repo to your local repository in any of your working branch.
● Once code is copied , go to the Build directory and build docker image from docker file and add meaningful tags and push to docker hub repository.
● Once Images are pushed to Docker hub registries, create a directory named kube.
Inside the kube directory create deployement.yaml file with 3 replication , label app: simpleapp-webapp , containerPort: 80 and add the image that you have pushed in Docker Hub registry.
● Create one service.yaml file with type nodeport & select simpleapp-webapp pod with port 80 & targetPort 80 with any nodePort between range 30000-32768.
● Open the webpage in the browser and notice the changes and capture the snap.
● Then delete the deployment that you have just created.
● Update the deployment.yaml file and add volumeMounts with mountPath /usr/share/nginx/html from emptyDir: {} volume.
● Once above changes are added, add initContainers block with below parameters. Also add volumeMounts for Init Container with mountPath "/work-dir" from emptyDir: {} volume.
initContainers:

- name: install
  image: busybox:1.28
  command:
- wget
- "-O"
- "/work-dir/index.html"
- http://info.cern.ch
  volumeMounts:
- name: workdir
  mountPath: "/work-dir"
  ● Add volumes with emptyDir: {} in deployment.yaml file.
  ● Once the deployment.yaml file is ready, create the deployment & access the page in the browser and notice the changes.

```
avej_lanjekar@AvejLanjekar:simpleapp$ ls
Dockerfile  html  wrapper.sh

avej_lanjekar@AvejLanjekar:simpleapp$ mkdir kube

avej_lanjekar@AvejLanjekar:simpleapp$ cd kube/

avej_lanjekar@AvejLanjekar:kube$ cat deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: init-cont-deplo
spec:
  replicas: 3
  selector:
    matchLabels:
      app: simpleapp-webapp

  template:
    metadata:
      labels:
        app: simpleapp-webapp
    spec:
      containers:
        - name: init-cont
          image: avejlanjekar45/cloudethix_initcontainer_avej
          ports:
            - containerPort: 80


avej_lanjekar@AvejLanjekar:kube$ cat service.yml
apiVersion: v1
kind: Service
metadata:
  name: init-service
spec:
  type: NodePort
  selector:
    app: simpleapp-webapp
  ports:
    - port: 80
      targetPort: 80
      nodePort: 32000

avej_lanjekar@AvejLanjekar:kube$ kubectl apply -f service.yml
service/init-service created
```

```
avej_lanjekar@AvejLanjekar:kube$ kubectl get svc init-service
NAME           TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
init-service   NodePort   10.99.78.194   <none>        80:32000/TCP   9s

avej_lanjekar@AvejLanjekar:kube$ minikube ip
192.168.49.2

avej_lanjekar@AvejLanjekar:kube$ curl http://192.168.49.2:32000
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Sample Deployment</title>
  <style>
    body {
      color: #ffffff;
      background-color: blue;
      font-family: Arial, sans-serif;
      font-size: 14px;
    }

    h1 {
      font-size: 500%;
      font-weight: normal;
      margin-bottom: 0;
    }

    h2 {
      font-size: 200%;
      font-weight: normal;
      margin-bottom: 0;
    }
  </style>
</head>
<body>
  <div align="center">
    <h1>Welcome to V1 of the web application</h1>
    <h2>This application will be deployed on Kubernetes.</h2>
  </div>
</body>
</html>

avej_lanjekar@AvejLanjekar:kube$ kubectl delete -f deployment.yml
deployment.apps "init-cont-deplo" deleted from default namespace
```

```
avej_lanjekar@AvejLanjekar:kube$ vim deployment.yml
avej_lanjekar@AvejLanjekar:kube$ cat deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: init-cont-deplo
spec:
  replicas: 3
  selector:
    matchLabels:
      app: simpleapp-webapp

  template:
    metadata:
      labels:
        app: simpleapp-webapp
    spec:
      initContainers:
        - name: install
          image: busybox:1.28
          command:
            - wget
            - -O
            - /work-dir/index.html
            - http://info.cern.ch
          volumeMounts:
            - name: workdir
              mountPath: /work-dir

      containers:
        - name: init-cont
          image: avejlanjekar45/cloudethix_initcontainer_avej
          ports:
            - containerPort: 80
          volumeMounts:
            - name: workdir
              mountPath: /usr/share/nginx/html
      volumes:
        - name: workdir
          emptyDir: {}

avej_lanjekar@AvejLanjekar:kube$ kubectl apply -f deployment.yml
deployment.apps/init-cont-deplo created

avej_lanjekar@AvejLanjekar:kube$ curl http://192.168.49.2:32000
<html><head></head><body><header>
```

```
<title>http://info.cern.ch</title>
</header>

<h1>http://info.cern.ch - home of the first website</h1>
<p>From here you can:</p>
<ul>
<li><a href="http://info.cern.ch/hypertext/WWW/TheProject.html">Browse the
first website</a></li>
<li><a href="http://line-
mode.cern.ch/www/hypertext/WWW/TheProject.html">Browse the first website using
the line-mode browser simulator</a></li>
<li><a href="http://home.web.cern.ch/topics/birth-web">Learn about the birth
of the web</a></li>
<li><a href="http://home.web.cern.ch/about">Learn about CERN, the physics
laboratory where the web was born</a></li>
</ul>
</body></html>
```

Q6

Create 1 Public Docker Hub registry named cloudethix_hpa_yourname.
● Clone below repository on your system.
https://github.com/vivekamin/kubernetes-hpa-example.git
● Initialize a local repository & copy the code from above repo to your local repository in any of your working branch.
● Once code is copied , build a docker image from the docker file and add meaningful tags and push to the docker hub repository.
● Once the image is pushed, go to k8s directory and update deployment.yaml file with image name from your repo. And then create it.
● Open service.yml and change the type to nodePort and apply the same.
● Open the HPA.yaml file, notice it and then apply the same.
● Open the browser, and access the webpage.
● Now it's time to test the HPA working with the below command.
kubectl run -i --tty load-generator --rm --image=busybox --restart=Never --
/bin/sh -c "while sleep 0.01; do wget -q -O- http://NODE_PORT_SERVICE_NAME;
done"
● Check the HPA from kubectl command and also check if the deployment is scaling up

```
avej_lanjekar@AvejLanjekar:kubernetes-hpa-example$ ls
Dockerfile  README.md  k8s  package.json  src
```

```
avej_lanjekar@AvejLanjekar:kubernetes-hpa-example$ cd k8s/

avej_lanjekar@AvejLanjekar:k8s$ ls
deployment.yml  hpa.yml  service.yml

avej_lanjekar@AvejLanjekar:k8s$ cat deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: node-example
spec:
  replicas: 1
  template:
    metadata:
      labels:
        app: node-example
    spec:
      containers:
      - name: node-example
        image: avejlanjekar45/cloudethix_hpa_avej
        imagePullPolicy: Always
        ports:
        - containerPort: 3000
        resources:
            limits:
              cpu: "0.5"
            requests:
              cpu: "0.25"

avej_lanjekar@AvejLanjekar:k8s$ cat service.yml
apiVersion: v1
kind: Service
metadata:
  name: node-example
  labels:
    app: node-example
spec:
  selector:
    app: node-example
  ports:
  - port: 3000
    protocol: TCP
    nodePort: 30001
  type: NodePort

avej_lanjekar@AvejLanjekar:k8s$ kubectl apply -f service.yml
```

```
service/node-example created

avej_lanjekar@AvejLanjekar:k8s$ cat hpa.yml
apiVersion: autoscaling/v1
kind: HorizontalPodAutoscaler
metadata:
  annotations:
  name: node-example
  namespace: default
spec:
  maxReplicas: 4
  minReplicas: 1
  scaleTargetRef:
    apiVersion: extensions/v1
    kind: Deployment
    name: node-example
  targetCPUUtilizationPercentage: 1

avej_lanjekar@AvejLanjekar:k8s$ kubectl apply -f hpa.yml
horizontalpodautoscaler.autoscaling/node-example created

avej_lanjekar@AvejLanjekar:k8s$ curl http://192.168.49.2:30001
Hello World!!

kubectl run -i --tty load-generator --rm --image=busybox --restart=Never --
/bin/sh -c "while sleep 0.01; do wget -q -O- http://node-example;
done"
/ # All commands and output from this session will be recorded in container
logs, including credentials and sensitive information passed through the
command prompt.
If you don't see a command prompt, try pressing enter.
/ #

avej_lanjekar@AvejLanjekar:~$ kubectl get hpa -w
NAME           REFERENCE                 TARGETS      MINPODS   MAXPODS
REPLICAS   AGE
node-example   Deployment/node-example   cpu: 0%/1%   1         4         1
4m40s
node-example   Deployment/node-example   cpu: 0%/1%   1         4         1
5m5s
```

Q7 →

- Create 1 Public Docker Hub registry named cloudethix_cronjob_yourname.
- Initialize a local repository & copy below code (three files) to your local repository in any

of your working branch.
- Once code is copied, build the docker image from Dockerfile , add meaningful tags and then push the docker image to Docker hub registry.
- Now update the pythoncronjob.yml file to change the image name that you have just pushed to docker hub registry.
- Now create a cron job using pythoncronjob.yml file. Check with kubectl command if the cron job is created.
- Check the Job name which is created by cronjob from command line or lens.
- Then check the pod logs which are created by the job and capture the output

```
avej_lanjekar@AvejLanjekar:cloudethix_cronjob$ ls
Dockerfile  helloworld.py  pythoncronjob.yml

avej_lanjekar@AvejLanjekar:cloudethix_cronjob$ cat Dockerfile
FROM python:3.7-alpine
#add user group and ass user to that group
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
#creates work dir
WORKDIR /app
#copy python script to the container folder app
COPY helloworld.py /app/helloworld.py
RUN chmod +x /app/helloworld.py
#user is appuser
USER appuser
ENTRYPOINT ["python", "/app/helloworld.py"]


avej_lanjekar@AvejLanjekar:cloudethix_cronjob$ cat helloworld.py
#!/usr/local/bin/python3
import datetime
x = datetime.datetime.now()
print("Welcome to the Cloudethix World")
print("Today is")
print(x)

avej_lanjekar@AvejLanjekar:cloudethix_cronjob$ cat pythoncronjob.yml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: python-helloworld
spec:
  schedule: "*/1 * * * *"
  jobTemplate:
    spec:
      template:
```

```
        spec:
          containers:
            - name: python-helloworld
              image: avejlanjekar45/cloudethix_cronjob_avej
              command: ["python","/app/helloworld.py"]
          restartPolicy: OnFailure

avej_lanjekar@AvejLanjekar:cloudethix_cronjob$ docker build -t cronjob .
STEP 1/7: FROM python:3.7-alpine
Resolved "python" as an alias
(/etc/containers/registries.conf.d/shortnames.conf)
Trying to pull docker.io/library/python:3.7-alpine...
Getting image source signatures
Copying blob 96526aa774ef done   |
Copying blob ea1518237b37 done   |
Copying blob 9875af95546d done   |
Copying blob 4819c95424fc done   |
Copying blob 148762f75a1f done   |
Copying config 1bac8ae77e done   |
Writing manifest to image destination
STEP 2/7: RUN addgroup -S appgroup && adduser -S appuser -G appgroup
--> 38b117328542
STEP 3/7: WORKDIR /app
--> 6438ab034fa9
STEP 4/7: COPY helloworld.py /app/helloworld.py
--> 63e924043b9b
STEP 5/7: RUN chmod +x /app/helloworld.py
--> a92c7ffe4a8c
STEP 6/7: USER appuser
--> 07de10ce0353
STEP 7/7: ENTRYPOINT ["python", "/app/helloworld.py"]
COMMIT cronjob
--> c41e700fdbd7
Successfully tagged localhost/cronjob:latest
c41e700fdbd76e0d5f291dbc4565caeec8cc244dfbddd17cf411ef37ac23c0ac


avej_lanjekar@AvejLanjekar:cloudethix_cronjob$ docker tag cronjob
avejlanjekar45/cloudethix_cronjob_avej

avej_lanjekar@AvejLanjekar:cloudethix_cronjob$ docker push
avejlanjekar45/cloudethix_cronjob_avej
Getting image source signatures
Copying blob 5795f33d6724 done   |
Copying blob 148762f75a1f skipped: already exists
Copying blob ea1518237b37 skipped: already exists
```

```
Copying blob 4819c95424fc skipped: already exists
Copying blob 734bd37c36be done   |
Copying blob 96526aa774ef skipped: already exists
Copying blob 734bd37c36be done   |
Copying blob 9875af95546d skipped: already exists
Copying config c41e700fdb done   |
Writing manifest to image destination

avej_lanjekar@AvejLanjekar:cloudethix_cronjob$ kubectl apply -f
pythoncronjob.yml
cronjob.batch/python-helloworld created



avej_lanjekar@AvejLanjekar:cloudethix_cronjob$ kubectl get jobs -w
NAME                          STATUS    COMPLETIONS   DURATION   AGE
python-helloworld-29807008   Running   0/1                      0s
python-helloworld-29807008   Running   0/1           0s         0s
python-helloworld-29807008   SuccessCriteriaMet   0/1           18s        18s
python-helloworld-29807008   Complete             1/1           17s        18s
python-helloworld-29807009   Running              0/1                      0s
python-helloworld-29807009   Running              0/1           0s         0s
python-helloworld-29807009   SuccessCriteriaMet   0/1           5s         5s
python-helloworld-29807009   Complete             1/1           5s         5s
python-helloworld-29807010   Running              0/1                      0s
python-helloworld-29807010   Running              0/1           0s         0s
python-helloworld-29807010   SuccessCriteriaMet   0/1           6s         6s
python-helloworld-29807010   Complete             1/1           6s         6s


avej_lanjekar@AvejLanjekar:cloudethix_cronjob$ kubectl get pods
NAME                               READY   STATUS                  RESTARTS
AGE
python-helloworld-29807008-fd8kr   0/1     Completed               0
2m46s
python-helloworld-29807009-frxgq   0/1     Completed               0
106s
python-helloworld-29807010-kxb9m   0/1     Completed               0
46s

avej_lanjekar@AvejLanjekar:cloudethix_cronjob$ kubectl get jobs
NAME                          STATUS     COMPLETIONS   DURATION   AGE
python-helloworld-29807009   Complete   1/1           5s         2m14s
python-helloworld-29807010   Complete   1/1           6s         74s
python-helloworld-29807011   Complete   1/1           6s         14s
avej_lanjekar@AvejLanjekar:cloudethix_cronjob$
```